

Error Handling In RESTful APIs

 <https://github.com/learn-co-curriculum/python-p4-rest-error-handling>  <https://github.com/learn-co-curriculum/python-p4-rest-error-handling/issues/new>

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.
- Determine the significant differences between 100-500 HTTP status codes.
- Determine best practices in error handling in Flask.

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

Try as we might, it is impossible to build a robust API without the possibility of errors and exceptions. Common issues include:

1. Creating records that already exist.
2. Creating, retrieving, updating, or deleting resources while your database server is down.
3. Creating or updating records with invalid attributes.
4. Retrieving, updating, or deleting records that don't exist.

The guiding principle of the REST philosophy is that the behavior of an API should be predictable. So how do we handle these cases outside of the functionality of our API in a predictable way?

HTTP Error Codes

Just like request methods, HTTP also specifies a number of response status codes to help the client understand if and why their request was successful. We've already talked about some status codes in earlier modules:

1. 200 (OK) means that the request was successful. This can be used for any request method, though it is not always the most descriptive. That being said, the client will be less interested in descriptive codes when requests are successful.
2. 201 (Created) means that the request was successful in creating a new resource. This is the best code to use after a successful **POST** request.
3. 404 (Not Found) means that the request was unsuccessful because it attempted to access a resource that does not exist on the server. After 200, this is the most common status code in a well-built API.

Codes are grouped by the hundreds:

- **100s** are informational. They inform the client that some process is being carried out before a final response is sent.
- **200s** denote a successful request.
- **300s** let the client know that a redirect is taking place. This is usually because a resource has been moved- *you should avoid doing this in RESTful APIs*.
- **400s** are client errors. They let the client know that *they* made a mistake.

- **500s** are server errors. They let the client know that something is temporarily out of order on your server.
- *Would a 400- or 500-class code be more appropriate for a client requesting a non-existent resource?*

Let's explore the error codes a bit further:

400-Class Codes: Client Errors

400-class status codes are intended as responses to client errors. These codes can be sent in response to any request method.

Despite their error, it is still our duty as developers (especially RESTful API developers) to give the client an informative message describing what went wrong.

The most common 400-class response codes are:

- **400 (Bad Request)**: the server is unable or not permitted to process the request, usually due to metadata in messages or forms.
- **401 (Unauthorized) and 403 (Forbidden)**: the user has not logged in *or* has logged in but is not permitted to access the resource.
- **404 (Not Found)**: as mentioned above, this means that the resource being sought after by the client does not exist.

For more information on 400-class codes, check out the [Mozilla documentation](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses)  [_\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses).

500-Class Codes: Server Errors

500-class status codes are acknowledgements that the server has produced an error. It is *critically important* to let the client know that a 500-class error has occurred and whether it is due to regular maintenance, a bug, or a permanent change.

500 (Internal Server Error) is by far the most common 500-class error. It can be appropriately used for nearly every server-side error, and with an appropriate and informative message, it is more than sufficient to meet the client's needs.

That being said, there are several other 500-class status codes for specific cases. For more information on 500-class codes, check out the [Mozilla documentation](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses)  [_\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses).

Error Handling in Flask

When an error occurs in the runtime of a Flask application, the corresponding status code will be returned to the client. As we've seen in previous lessons, you can also easily set the status code in the construction of a response: it is the optional second argument of the `make_response()` function. When one of these errors takes place, it is most useful to return a message specific to the application. This is most easily carried out through registering **error handlers**.

Registering Error Handlers

Similarly to how flask registers routes, error handlers are registered through use of the `@app.errorhandler()` decorator. Responses are constructed the same as with any other view, either as comma-separated body, status code, headers or an object created with `make_response()`. (The latter is preferred.)

Let's take a look at a handler for 404s:

```
# example
from werkzeug.exceptions import NotFound

...

@app.errorhandler(NotFound)
def handle_not_found(e):

    response = make_response(
        "Not Found: The requested resource does not exist.",
        404
    )

    return response

app.register_error_handler(404, handle_not_found)
```

This will automatically capture any 404s detected by your Flask application and return the message provided in the response body. Flask can accomplish this because of the `werkzeug.exceptions.NotFound` class- this is the exception that is raised when a 404 is detected, and we have registered it to our handler. The two are now partnered for life!

`werkzeug.exceptions` [↗\(https://werkzeug.palletsprojects.com/en/2.2.x/exceptions/\)](https://werkzeug.palletsprojects.com/en/2.2.x/exceptions/) includes all HTTP 400-class and 500-class codes, and you should think about which are relevant to your API as you build. An HTTP error should never go unhandled, or the client will not know how to proceed with their work.

IMPORTANT: Make sure the messages in your response bodies are as specific as possible! Imagine if Python applications only told you "something went wrong" when there was an error in your code. You'd never figure it out!

► *Why is it considered a best practice to provide specific error messages when building APIs?*

Conclusion

Even the best APIs throw errors- even 500-class errors- sometimes. In order to avoid too many GitHub issues (or worse- people giving up on your API), it's important to register error handlers for every 400- or 500-class status code that might be returned by your API.

Flask makes registration of error handlers very simple through use of the `app.errorhandler()` decorator. Feeding this the error code exception from `werkzeug.exceptions` and generating a descriptive response are all you need to do to elevate a simple API to a simple *production quality* API.

Resources

- [Handling Application Errors - Pallets](https://flask.palletsprojects.com/en/2.2.x/errorhandling/) [↗\(https://flask.palletsprojects.com/en/2.2.x/errorhandling/\)](https://flask.palletsprojects.com/en/2.2.x/errorhandling/)
- [Client error responses - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses) [↗\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#client_error_responses)
- [Server error responses - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses) [↗\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#server_error_responses)
- [8. Errors and Exceptions - Python](https://docs.python.org/3.8/tutorial/errors.html) [↗\(https://docs.python.org/3.8/tutorial/errors.html\)](https://docs.python.org/3.8/tutorial/errors.html)

- [HTTP Exceptions - Pallets](https://werkzeug.palletsprojects.com/en/2.2.x/exceptions/)  [\(https://werkzeug.palletsprojects.com/en/2.2.x/exceptions/\)](https://werkzeug.palletsprojects.com/en/2.2.x/exceptions/)